

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería
Ingeniería de Sistemas e Informática

Herramientas de Desarrollo Profesional - TIC
Unidad 2: Prompt Engineering e Inteligencia Artificial
Sesión 2: Creación de Chatbots y aplicación de modelos de OpenAI

Práctica: Chatbots multimodales Chatbot de comida peruana

Python, Streamlit, API de chat compatible con OpenAI y Whisper

Integrantes

Leonardo Martinez Concha
Kevin Julio Chacaltana Vargas

Septiembre de 2026

1. Descripción del proyecto

Esta práctica consistió en construir un chatbot que responde preguntas sobre la comida peruana. El trabajo se dividió en dos partes. En la primera se implementó un chat de texto que conversa con un modelo de lenguaje a través de una API y recuerda el historial de la sesión. En la segunda se agregó la entrada por voz: el usuario graba una pregunta con el micrófono o sube un archivo de audio, el modelo Whisper lo convierte a texto y esa transcripción entra al mismo flujo del chat.

La interfaz se hizo con Streamlit. El área principal muestra la conversación con los componentes de chat de la librería y la barra lateral concentra la configuración y los controles de audio. El chatbot tiene una personalidad definida en el prompt de sistema: se llama Sazón, habla como un cocinero peruano y solo responde temas de gastronomía del Perú. Si alguien le pregunta algo distinto, lo redirige con amabilidad hacia el tema.

2. Herramientas y decisión sobre la API

La sesión de clase usa la API de OpenAI, que es de pago. Para no depender de una tarjeta de crédito se buscó una alternativa gratuita que permitiera reutilizar el mismo código. Google Gemini ofrece una capa gratuita en AI Studio y expone un endpoint compatible con la API de OpenAI. Gracias a eso el chatbot usa la librería oficial `openai` de Python vista en clase y solo cambian dos cosas: la URL base y la clave. La capa gratuita de Gemini tiene un tope de peticiones por día y por modelo (20 al día en el caso de `gemini-3.6-flash`), suficiente para una demostración. Como cada modelo tiene su propia cuota, el código define una lista de modelos de respaldo: si el principal devuelve el error 429, la función `responder()` prueba el siguiente y la interfaz avisa qué modelo contestó. El proveedor y el modelo también quedaron configurables en el archivo `.env`: con una clave gratuita de Groq el mismo código atiende miles de peticiones diarias.

Para la voz se decidió ejecutar Whisper en la propia computadora con la librería `faster-whisper`, que corre el modelo de OpenAI sobre CPU sin necesidad de clave. El modelo se descarga una sola vez, cuando se hace la primera transcripción. El código también contempla la ruta por API: si en el archivo `.env` existe una clave de Groq, la transcripción se envía al endpoint de audio de Groq, que es idéntico al de OpenAI.

Herramienta	Versión o modelo	Uso en el proyecto
Python	3.12.10	Lenguaje del proyecto
Streamlit	1.64.0	Interfaz web del chatbot (chat, barra lateral, carga de audio, grabación)

Librería openai	3.15.0	Cliente para el endpoint de chat completions (y el de audio, si se usa Groq)
Google Gemini API (gratuita)	gemini-3.5-flash-lite	Modelo de lenguaje que responde las preguntas (Parte 1)
faster-whisper	1.2.1	Ejecuta el modelo Whisper "small" de OpenAI en la computadora (Parte 2)
python-dotenv	1.2.3	Lectura de la clave desde el archivo .env
truststore	0.10.4	Hace que Python use los certificados del sistema (evita errores SSL con antivirus o proxies)
edge-tts	7.2.8	Generación de los audios de prueba con voz peruana (solo para las pruebas)

Durante las pruebas apareció un error de certificados SSL. La causa era el antivirus de la computadora, que inspecciona el tráfico HTTPS con un certificado propio que Python no reconocía. Se resolvió con la librería truststore, que hace que Python confíe en los mismos certificados que el sistema operativo.

3. Cómo funciona

Parte 1: chatbot de texto.

1. Streamlit guarda el historial en `st.session_state.mensajes` y lo dibuja con `st.chat_message` en cada ejecución del script.
2. Cuando el usuario escribe en `st.chat_input`, la pregunta se agrega al historial.
3. La función `responder()` arma la lista de mensajes con el prompt de sistema más todo el historial y llama a `chat.completions.create` con `stream=True`.
4. La respuesta llega por partes y se muestra en vivo con `st.write_stream`. Al terminar se guarda en el historial, de modo que el modelo mantiene el contexto en las siguientes preguntas.

Parte 2: transcripción con Whisper.

1. En la barra lateral, `st.audio_input` graba desde el micrófono y `st.file_uploader` permite subir un archivo mp3, wav, m4a, ogg, webm o flac.
2. Al pulsar el botón, los bytes del audio pasan a la función `transcribir()`. Whisper recibe el audio en memoria, el idioma español y un texto de contexto con nombres de platos que le ayuda a escribirlos bien. El modelo se carga una sola vez y queda en memoria para las siguientes preguntas.
3. El texto transcrito se muestra en la barra lateral y se envía al chatbot como si el usuario lo hubiera escrito. En el chat aparece con un ícono de micrófono.

Estructura del repositorio:

```
PROYECTOCHATBOT/  
├─ app.py           # Interfaz en Streamlit (chat + entrada por voz)  
├─ chatbot.py       # Lógica: cliente, respuesta del modelo y transcripción con  
Whisper  
├─ requirements.txt  # Dependencias  
├─ .env.example      # Plantilla de variables de entorno  
├─ .streamlit/config.toml # Tema de la interfaz  
├─ audio_prueba/     # Audios de ejemplo para probar la Parte 2  
├─ docs/             # Capturas de la ejecución y este PDF  
└─ README.md
```

4. Código fuente

app.py

Interfaz del chatbot en Streamlit (Parte 1 y Parte 2)

```
"""
Chatbot de Comida Peruana - interfaz en Streamlit.

Parte 1: chat de texto con un modelo de lenguaje.
Parte 2: preguntas por voz (micrófono o archivo) transcritas con Whisper.
"""

import streamlit as st

from chatbot import (
    MODELO_CHAT,
    MODELO_WHISPER,
    NOMBRE_PROVEEDOR,
    crear_cliente,
    responder,
    transcribir,
)

st.set_page_config(page_title="Chatbot de Comida Peruana", page_icon="🍽️")

BIENVENIDA = (
    "¡Hola! Soy **Sazón**, tu guía de comida peruana. "
    "Pregúntame por platos, ingredientes, recetas, historia o qué comer en cada región del Perú."
)

# ----- Estado de la sesión (historial de la conversación) -----
if "mensajes" not in st.session_state:
    st.session_state.mensajes = [{"role": "assistant", "content": BIENVENIDA}]

def reiniciar_chat():
    st.session_state.mensajes = [{"role": "assistant", "content": BIENVENIDA}]

# ----- Barra lateral: configuración y entrada por voz -----
with st.sidebar:
    st.header("⚙️ Configuración")
    api_key = st.text_input(
        "API key (opcional si está en .env)",
        type="password",
        help="La clave de Gemini se obtiene gratis en https://aistudio.google.com/apikey",
    )
    st.caption(f"Proveedor: {NOMBRE_PROVEEDOR}")
    st.caption(f"Modelo de chat: `{MODELO_CHAT}`")
    st.caption(f"Modelo de voz: `{MODELO_WHISPER}`")
    st.button("🗣️ Nueva conversación", on_click=reiniciar_chat)

st.divider()
st.header("🎧 Parte 2: pregunta por voz")
grabacion = st.audio_input("Graba tu pregunta")
archivo = st.file_uploader(
    "...o sube un archivo de audio",
    type=["mp3", "wav", "m4a", "ogg", "webm", "flac"],
)
```

```

    )
    boton_transcribir = st.button("🗣️ Transcribir con Whisper y enviar")

# ----- Zona principal: el chat -----
st.title("🍽️ Chatbot de Comida Peruana")
st.caption("Práctica: Chatbots multimodales · Python + Streamlit + modelo de lenguaje + Whisper")

for mensaje in st.session_state.mensajes:
    with st.chat_message(mensaje["role"]):
        prefijo = "🗣️ " if mensaje.get("por_voz") else ""
        st.markdown(prefijo + mensaje["content"])

def enviar(texto: str, por_voz: bool = False):
    """Agrega la pregunta al historial, consulta al modelo y muestra la respuesta."""
    st.session_state.mensajes.append({"role": "user", "content": texto, "por_voz": por_voz})
    with st.chat_message("user"):
        st.markdown(("🗣️ " if por_voz else "") + texto)

    with st.chat_message("assistant"):
        try:
            cliente = crear_cliente(api_key)
            info = {}
            respuesta = st.write_stream(responder(cliente, st.session_state.mensajes, info))
            if info.get("modelo") and info["modelo"] != MODELO_CHAT:
                st.caption(f"Respondió el modelo de respaldo `{info['modelo']}` (cuota del principal agotada).")
        except Exception as error:
            respuesta = f"⚠️ No pude responder: {error}"
            st.error(respuesta)
        st.session_state.mensajes.append({"role": "assistant", "content": respuesta})

# Parte 2: audio -> Whisper -> texto -> chatbot
if boton_transcribir:
    audio = grabacion or archivo
    if audio is None:
        st.sidebar.warning("Primero graba o sube un audio.")
    else:
        with st.spinner("Transcribiendo con Whisper (la primera vez carga el modelo)..."):
            try:
                texto = transcribir(audio.getvalue(), audio.name)
            except Exception as error:
                texto = ""
                st.sidebar.error(f"No pude transcribir: {error}")
        if texto:
            st.sidebar.success(f"Transcripción: {texto}")
            enviar(texto, por_voz=True)

# Parte 1: texto escrito
if pregunta := st.chat_input("Escribe tu pregunta sobre comida peruana..."):
    enviar(pregunta)

```

chatbot.py

Lógica del chatbot: cliente, respuesta del modelo y transcripción con Whisper

```

"""
Lógica del chatbot de comida peruana.

```

Parte 1: responder preguntas con un modelo de lenguaje usando la librería oficial `openai`. Por defecto se conecta a Google Gemini (capa gratuita), que expone una API compatible con la de OpenAI: el código es el mismo, solo cambian la URL base y la clave.

Parte 2: transcribir audios con el modelo Whisper. Por defecto Whisper corre en la propia computadora (faster-whisper, sin clave). Si existe GROQ_API_KEY, se usa el endpoint de transcripciones de Groq, idéntico al de OpenAI.

```
"""
import io
import os
from functools import lru_cache

from dotenv import load_dotenv
from openai import NotFoundError, OpenAI, RateLimitError

try: # Usa los certificados del sistema; evita errores SSL con antivirus o proxies
    import truststore
    truststore.inject_into_ssl()
except ImportError:
    pass

load_dotenv() # carga las variables definidas en el archivo .env

# ----- Parte 1: proveedor del modelo de lenguaje (API compatible con OpenAI) -----
NOMBRE_PROVEEDOR = os.getenv("PROVEEDOR", "Google Gemini (capa gratuita)")
BASE_URL = os.getenv("LLM_BASE_URL",
    "https://generativelanguage.googleapis.com/v1beta/openai/")
MODELO_CHAT = os.getenv("MODELO_CHAT", "gemini-3.5-flash-lite")
# La capa gratuita limita las peticiones por día y por modelo. Si el modelo principal
# agota su cuota (error 429), se prueban estos modelos en orden.
MODELOS_RESPALDO = [
    m.strip()
    for m in os.getenv("MODELOS_RESPALDO", "gemini-3.1-flash-lite,gemini-flash-lite-
latest,gemini-3.6-flash").split(",")
    if m.strip()
]
VARIABLES_CLAVE = ("LLM_API_KEY", "GEMINI_API_KEY", "GROQ_API_KEY", "OPENAI_API_KEY")

# ----- Parte 2: Whisper -----
WHISPER_LOCAL = os.getenv("WHISPER_LOCAL", "small") # tiny, base, small, medium o large-v3
WHISPER_API = os.getenv("MODELO_WHISPER", "whisper-large-v3-turbo")
USAR_WHISPER_API = bool(os.getenv("GROQ_API_KEY"))
MODELO_WHISPER = f"{WHISPER_API} (API de Groq)" if USAR_WHISPER_API else f"Whisper
{WHISPER_LOCAL} (local)"
VOCABULARIO = (
    "Pregunta sobre comida peruana: ceviche, lomo saltado, ají de gallina, "
    "causa limeña, anticuchos, rocoto relleno, chicha morada, pisco sour."
)

PROMPT_SISTEMA = """Eres "Sazón", un asistente experto en gastronomía peruana.
Tu única especialidad es la comida y la bebida del Perú: platos típicos, ingredientes,
recetas paso a paso, historia y origen de los platos, cocina por regiones (costa, sierra
y selva), postres, bebidas como la chicha morada o el pisco sour, y recomendaciones
de qué probar y dónde.

Reglas:
1. Responde siempre en español, con un tono cercano y amable, como un cocinero peruano.
2. Si la pregunta no tiene relación con la comida peruana, explica con amabilidad que
solo puedes hablar de gastronomía peruana y sugiere una pregunta relacionada.
3. Sé claro y ordenado: usa listas o pasos numerados cuando expliques recetas.
4. No inventes datos. Si no estás seguro de algo, dilo.
5. Mantén las respuestas breves (unas 200 palabras como máximo), salvo que pidan
```

```

una receta completa.
"""

def crear_cliente(api_key: str | None = None) -> OpenAI:
    """Crea el cliente. La clave puede venir de la interfaz o del archivo .env."""
    key = api_key or next((os.getenv(v) for v in VARIABLES_CLAVE if os.getenv(v)), None)
    if not key:
        raise ValueError(
            "No se encontró la API key. Escríbela en la barra lateral "
            "o guárdala en el archivo .env (por ejemplo GEMINI_API_KEY)."
        )
    return OpenAI(api_key=key, base_url=BASE_URL, max_retries=1)

def responder(cliente: OpenAI, historial: list[dict], info: dict | None = None):
    """Parte 1: devuelve la respuesta del modelo como un generador (streaming).

    Si el modelo principal responde con error 429 (cuota agotada) o no está disponible,
    se intenta con los modelos de respaldo. El modelo que respondió queda en info["modelo"].
    """
    mensajes = [{"role": "system", "content": PROMPT_SISTEMA}] + [
        {"role": m["role"], "content": m["content"]} for m in historial
    ]
    stream, ultimo_error = None, None
    for modelo in [MODELO_CHAT, *MODELOS_RESPALDO]:
        try:
            stream = cliente.chat.completions.create(
                model=modelo,
                messages=mensajes,
                temperature=0.7,
                max_tokens=2000,
                stream=True,
            )
            break
        except (RateLimitError, NotFoundError) as error: # probar el siguiente modelo
            ultimo_error = error
    if stream is None:
        raise ultimo_error
    if info is not None:
        info["modelo"] = modelo
    for parte in stream:
        if parte.choices and parte.choices[0].delta.content:
            yield parte.choices[0].delta.content

@lru_cache(maxsize=1)
def _cargar_whisper():
    """Carga el modelo Whisper local una sola vez (la primera vez lo descarga)."""
    from faster_whisper import WhisperModel

    return WhisperModel(WHISPER_LOCAL, device="cpu", compute_type="int8")

def transcribir(audio_bytes: bytes, nombre_archivo: str = "audio.wav") -> str:
    """Parte 2: convierte un audio a texto usando el modelo Whisper."""
    if USAR_WHISPER_API:
        cliente = OpenAI(api_key=os.getenv("GROQ_API_KEY"),
        base_url="https://api.groq.com/openai/v1")
        resultado = cliente.audio.transcriptions.create(
            model=WHISPER_API,
            file=(nombre_archivo, audio_bytes),

```



```

        language="es",
        prompt=VOCABULARIO,
        response_format="text",
    )
    texto = resultado if isinstance(resultado, str) else resultado.text
    return texto.strip()

segmentos, _ = _cargar_whisper().transcribe(
    io.BytesIO(audio_bytes), language="es", beam_size=5, initial_prompt=VOCABULARIO
)
return " ".join(segmento.text.strip() for segmento in segmentos)

```

requirements.txt

Dependencias del proyecto

```

streamlit>=1.47
openai>=1.50
python-dotenv>=1.0
faster-whisper>=1.1
truststore>=0.10

```

.env.example

Plantilla de variables de entorno (la clave real va en .env y no se sube al repositorio)

```

# Copia este archivo como .env y pega tu clave.
# Clave gratuita de Google Gemini: https://aistudio.google.com/apikey
GEMINI_API_KEY=pega_aqui_tu_clave

# Modelos que se usan, en orden, si el principal agota su cuota diaria gratuita
# MODELOS_RESPALDO=gemini-3.1-flash-lite,gemini-flash-lite-latest,gemini-3.6-flash

# Tamaño del modelo Whisper local (tiny, base, small, medium). Se descarga la primera vez.
# WHISPER_LOCAL=small

# Opcional: usar Groq en lugar de Gemini (clave gratuita en https://console.groq.com/keys).
# Si existe GROQ_API_KEY, la transcripción con Whisper se hace por la API de Groq.
# GROQ_API_KEY=gsk...
# LLM_API_KEY=gsk...
# LLM_BASE_URL=https://api.groq.com/openai/v1
# MODELO_CHAT=llama-3.3-70b-versatile
# PROVEEDOR=Groq (capa gratuita)

```

.streamlit/config.toml

Tema de colores de la interfaz

```

[theme]
primaryColor = "#D91023"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#FFF3F0"
textColor = "#262730"

```

5. Ejecución del chatbot en la interfaz de usuario

La aplicación se inicia con `streamlit run app.py` y se abre en el navegador en `http://localhost:8501`. Las capturas siguientes muestran las dos partes de la práctica funcionando.

Salida en consola al iniciar la aplicación

```
(.venv) C:\chatbot-comida-peruana> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
```

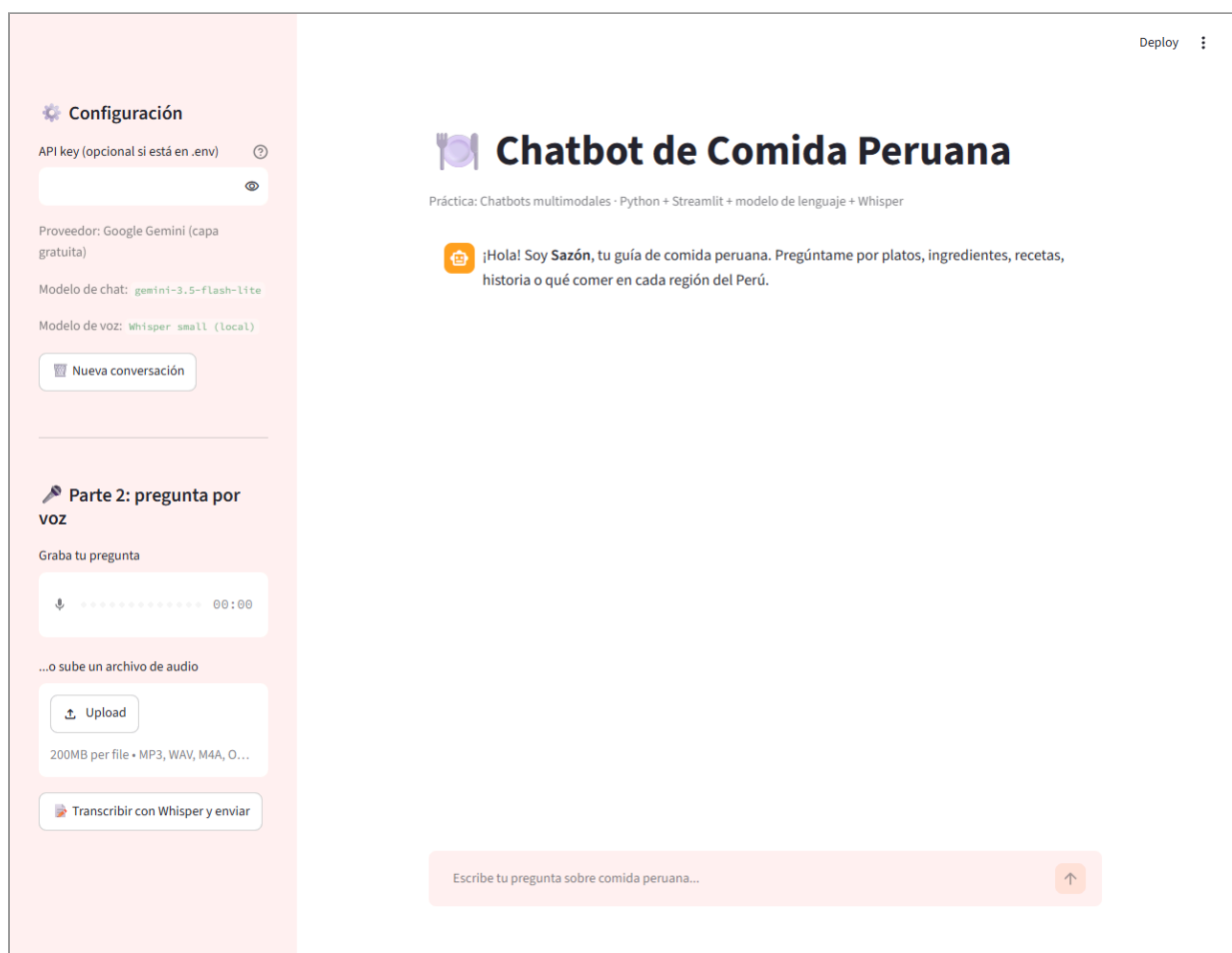


Figura 1. Pantalla inicial del chatbot. A la izquierda la barra lateral con la configuración y los controles de voz; a la derecha el mensaje de bienvenida.

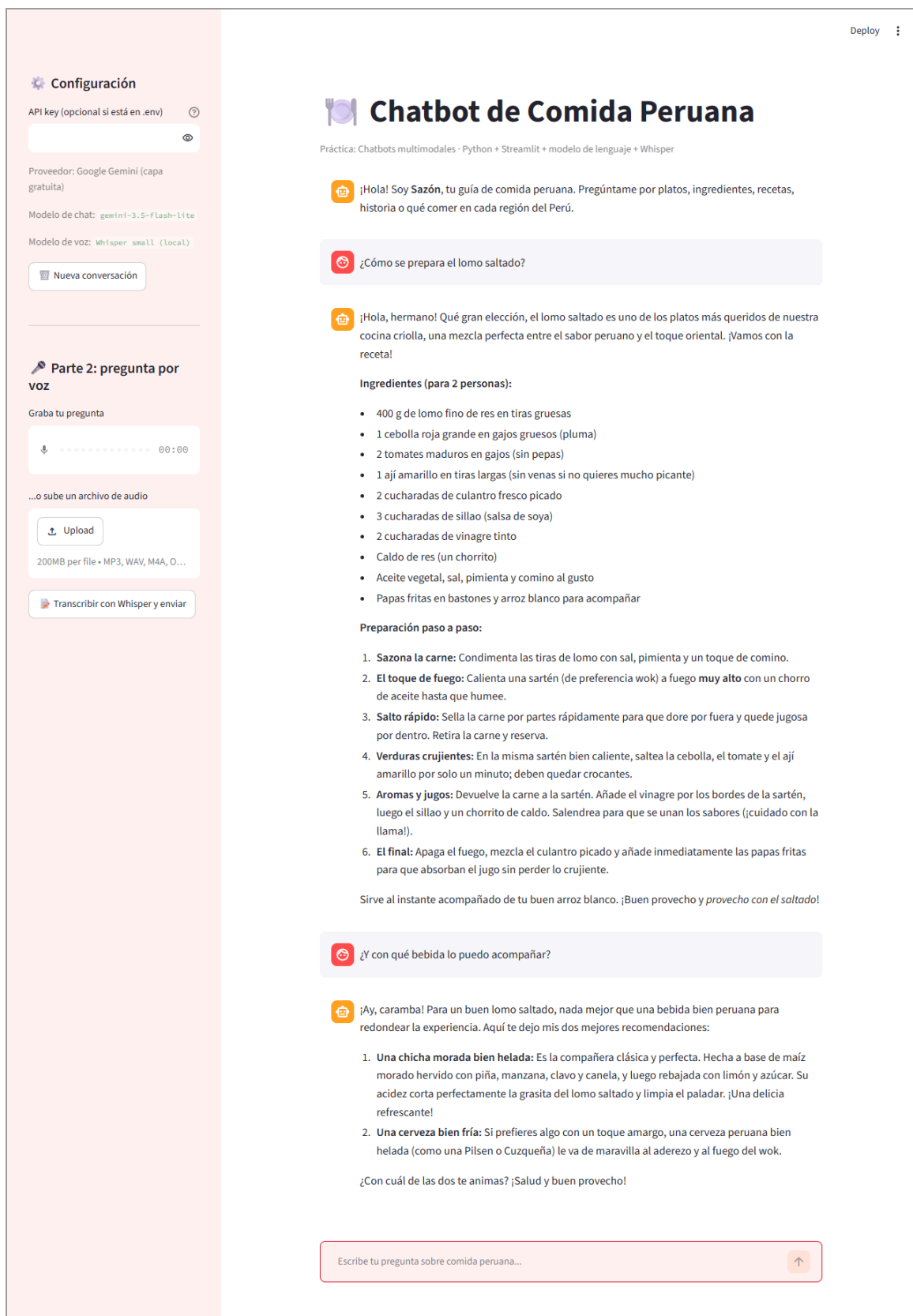


Figura 2. Parte 1: conversación de texto. La segunda pregunta no menciona el plato y el chatbot igual responde sobre el lomo saltado, porque recibe todo el historial de la sesión.

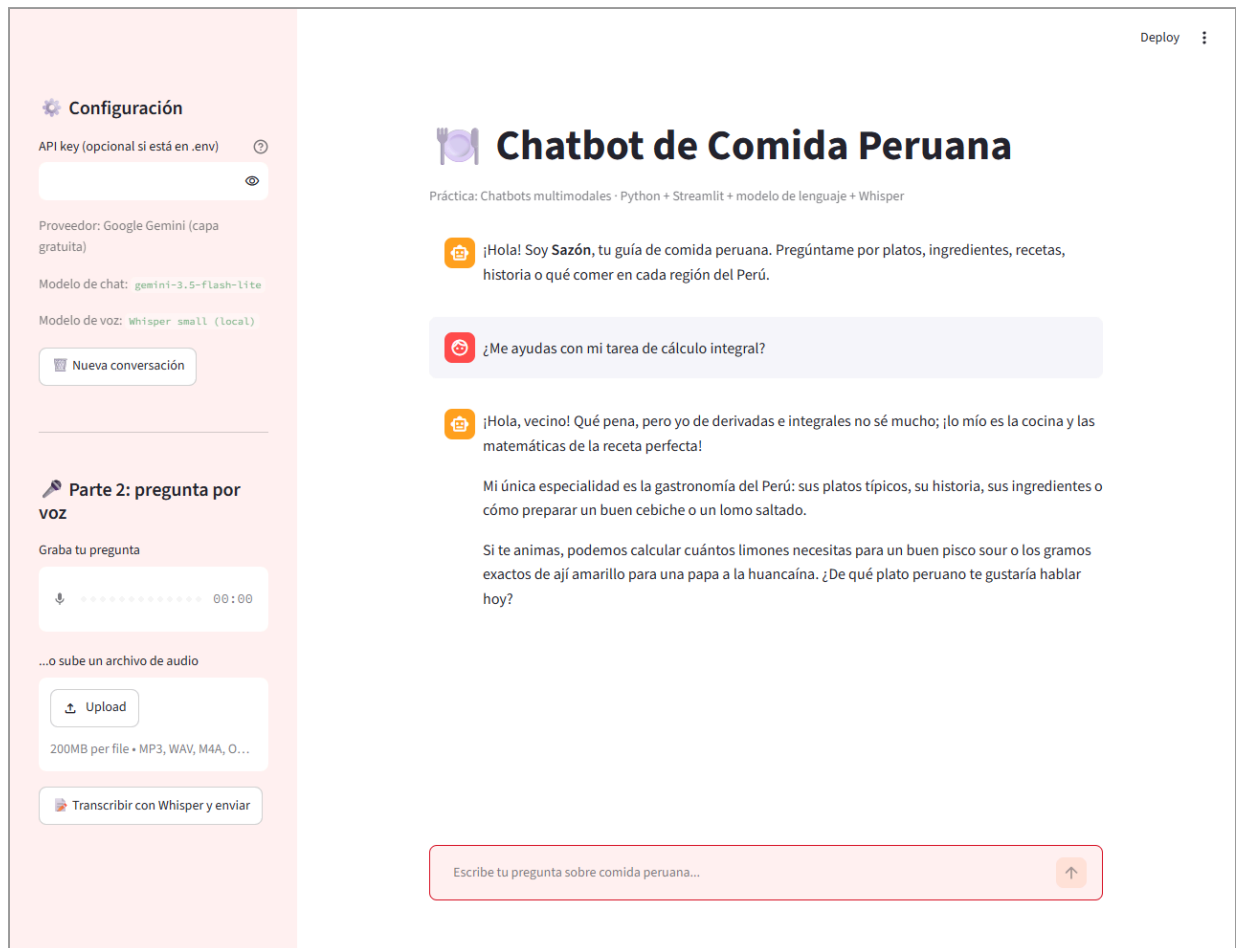


Figura 3. El prompt de sistema en acción: ante una pregunta que no trata de comida peruana, el chatbot lo explica y sugiere volver al tema.

Deploy

Configuración

API key (opcional si está en .env)

Proveedor: Google Gemini (capa gratuita)

Modelo de chat: gemini-3.5-flash-lite

Modelo de voz: Whisper small (local)

Nueva conversación

Parte 2: pregunta por voz

Graba tu pregunta

...o sube un archivo de audio

pregunta_ceviche.mp3
27.4KB

Transcribir con Whisper y enviar

Transcripción: Hola, ¿cómo se prepara un ceviche peruano y qué ingredientes necesito?

Chatbot de Comida Peruana

Práctica: Chatbots multimodales · Python + Streamlit + modelo de lenguaje + Whisper

¡Hola! Soy **Sazón**, tu guía de comida peruana. Pregúntame por platos, ingredientes, recetas, historia o qué comer en cada región del Perú.

Hola, ¿cómo se prepara un ceviche peruano y qué ingredientes necesito?

¡Hola, hermanito! Qué buen gusto tienes, el ceviche es el rey de nuestra costa y orgullo nacional. Prepararlo en casa es bien fácil si sigues estos pasos y usas productos fresquitos.

Aquí tienes los **ingredientes** para 4 porciones:

- 1 kilo de pescado fresco de carne firme (corvina, lenguado, perico o mero).
- 15 a 20 limones sutiles (¡ojo, limones peruanos, jugosos y ácidos!).
- 1 cebolla roja grande cortada en pluma fina.
- 1 cucharada de rocoto o ají limo picado en cuadraditos chiquitos (sin venas si no quieres que pique mucho).
- 1 cucharada de culantro (cilantro) fresco picado.
- Sal y pimienta al gusto.
- Para acompañar:** camote sancochado, choclo desgranado, yuyo (opcional) y cancha serrana.

Preparación **paso a paso**:

- Corta el pescado:** Lávalo bien y córtalo en cubos medianos (de unos 2 o 3 centímetros). Ponlo en un bol frío.
- Sazona:** Añade sal y pimienta al gusto. Remueve bien.
- El ají y el culantro:** Agrega el rocoto o ají limo y el culantro picado.
- Exprime el limón:** Hazlo directamente sobre el pescado, pero **nunca lo exprimas hasta el fondo** para que no amargue. Debe cubrir casi todo el pescado.
- Integra y reposa:** Mezcla suavemente por un minuto. Prueba la sazón. Añade la cebolla lavada previamente en agua fría.
- Sirve al toque:** Sírvelo inmediatamente acompañado de su camote, choclo, cancha y su buen chorrito de leche de tigre.

¡Y listo! A disfrutar. Si tienes otra duda sobre nuestra gastronomía, ¡aquí estoy!

Escribe tu pregunta sobre comida peruana...

Figura 4. Parte 2: se sube el audio pregunta_ceviche.mp3, Whisper lo transcribe (texto en verde en la barra lateral) y el chatbot responde a la pregunta transcrita, marcada con el ícono de micrófono.

6. Enlace del proyecto en el repositorio

El código completo, los audios de prueba y este documento están publicados en GitHub:

<https://github.com/Linkingfor/PROYECTOCHATBOT>

Pasos para ejecutarlo en otra computadora:

```
git clone https://github.com/Linkingfor/PROYECTOCHATBOT.git
cd PROYECTOCHATBOT
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env      (pegar la clave gratuita de Gemini dentro de .env)
streamlit run app.py
```

7. Conclusiones

El chatbot cumple las dos partes de la práctica: conversa sobre comida peruana manteniendo el contexto de la sesión y acepta preguntas por voz que Whisper transcribe antes de responder. Cambiar la API de OpenAI por Gemini no obligó a reescribir el código de clase, solo la URL base y la clave. Whisper, corriendo en local, transcribió bien los audios en español y reconoció los nombres de los platos. Como mejora futura se podría agregar respuesta por voz con un modelo de texto a voz, para que la conversación sea completamente hablada.